☕ Tap-a-Latte

Self-Ordering Kiosk & Cafe Management System for Coffee Shops in Kabacan, North Cotabato

CAPSTONE PROPOSAL DEFENSE CHEAT-SHEET & GUIDE

1. What is Tap-a-Latte? (Purpose & Objectives)

> 📌 **In Plain English for Your Group:**
>
> **Tap-a-Latte** is a smart coffee ordering system designed specifically for small and medium coffee shops in **Kabacan, North Cotabato**. It replaces traditional paper order pads with a modern digital ordering kiosk that customers can tap on, while simultaneously giving baristas and shop owners real-time management tools.

The Problems We Are Solving:

- **Long Waiting Times:** Customers waiting in long lines to order during rush hours (students, teachers, office workers).
- **Order Mistakes:** Cashiers mishearing drink specifications (wrong sugar level, wrong cup size, hot vs. iced mistakes).
- **Payment Delays:** Slow handling of cash and manual verification of GCash and PayMaya reference numbers.
- **Inventory Shortages:** Baristas suddenly discovering they have run out of coffee beans or milk in the middle of a rush.

Our Proposed Solution:

- A **Self-Service Kiosk** where customers choose and customize their drinks exactly how they want them.
- An **Automated Kitchen Display System (KDS)** where baristas see incoming drink recipes in real-time.
- An **Inventory Tracker** that automatically deducts beans, milk, syrups, and cups with each order and triggers low-stock alerts.

2. Technologies, Frameworks & Languages Used

Here is the exact tech stack. When the panel asks "*What technologies did you use and why?*", refer directly to this table:

CATEGORY	TECHNOLOGY USED	WHY WE USED IT (SIMPLE REASON)
Frontend Language	**TypeScript / JavaScript**	Catches code bugs before runtime; ensures type safety for drink orders and currency calculations.
UI Framework	**React 18 + Vite**	Renders pages instantly in 600ms; reactive components for live cart updates and instant drink customizations.

CATEGORY	TECHNOLOGY USED	WHY WE USED IT (SIMPLE REASON)
Styling / Design	**Tailwind CSS**	Allowed us to craft a custom artisan coffee color palette (espresso darks, caramel ambers, latte cream).
Mobile Engine (Android & iOS)	**Capacitor 8** (by Ionic)	Turns our responsive application into 100% native Android (APK) and Apple iOS (Xcode) apps without rewriting code twice.
Cross-Platform Alternative	**Flutter (Dart)**	Included in `flutter_app/` for cross-platform mobile native compiles.
Database Architecture	**Relational SQL (SQLite / MySQL)**	11 normalized tables with foreign keys and automatic inventory triggers; runs offline in the app and on server databases.
Backend API Architecture	**Laravel 11 (PHP 8.3)**	Includes full Eloquent Models and REST API routes in `laravel_backend/`.
Cloud Deployment	**Vercel + GitHub Actions**	Vercel hosts the web app 24/7 for free; GitHub Actions automatically builds the Android APK in the cloud.

3. How the App Works (Step-by-Step Flow)

Walk the panel through this simple 5-step scenario during your demo:

Step 1: Customer Browses & Customizes (Kiosk Ordering)

The customer walks up to the kiosk or opens the mobile app. They select a drink (e.g., *Kabacan Spanish Latte*) and customize it: **Sugar level (50%)**, **Cup size (Large 16oz)**, **Milk type (Oat Milk)**, and add-ons like **Extra Espresso Shot**. They choose **Dine-In** or **Take-Out**.

Step 2: Payment & Digital Ticket Generation

Customer proceeds to checkout. They select **GCash** (scans QR), **PayMaya** (scans QR), or **Cash at Counter**. Once confirmed, the system instantly prints/displays a **Digital Queue Ticket with a live QR Code** (e.g., `#LATT-101`).

Step 3: Cashier Verifies (POS Terminal)

If the customer chose Cash, the order enters the Cashier's pending queue. The cashier inputs the cash tendered (e.g., ₱500 bill), the system automatically calculates the change, and marks the ticket as **Paid**.

Step 4: Barista Prepares (Kitchen Display System - KDS)

The order immediately appears on the Barista's kitchen screen with exact recipe specifications. The barista clicks **"Start Brewing"**, then **"Mark as Ready for Pickup"** when the coffee is poured.

Step 5: Automatic Inventory Deduction & Admin Tracking

The system automatically deducts the raw ingredients (18g coffee beans, 200ml milk, 1 cup). If beans or milk fall below safety thresholds, the Owner/Admin dashboard immediately triggers a **Low Stock Alert**.

4. User Roles & Access Separation

Access is divided into 4 distinct portals, protected by quick PIN authentication:

ROLE	QUICK PIN	WHAT THEY CAN ACCESS / SEE
Customer	None (Open)	Menu browsing, Drink customizer, Dine-In/Take-Out, GCash/Maya/Cash checkout, Digital ticket.
Cashier	2222	Pending cash queue, GCash/Maya reference verification, Cash tender & change calculator, Print receipt.

ROLE	QUICK PIN	WHAT THEY CAN ACCESS / SEE
Staff / Barista	3333	Kitchen Display System (KDS Kanban), incoming drink recipes, one-click order status advancement.
Admin / Owner	1234	Executive Dashboard (Daily sales, total orders, AOV), User Management (CRUD accounts), Menu item CRUD & Availability toggle, Real-Time Ingredient Inventory.

5. Unique Feature: Automatic Dual-Layout (No Button)

> **Key Point for Panel:** Unlike basic websites that simply shrink, our system features **two completely separate layouts** that swap **100% automatically**:
>
> • **Dedicated Mobile Layout:** Triggers automatically on phones (width < 768px or vertical portrait orientation). Features thumb-friendly touch cards, bottom navigation, and slide-up sheets.
> • **Dedicated PC / Kiosk Split Layout:** Triggers automatically on wide monitors or landscape kiosk tablets (≥ 768px). Features a 3-column split view: Category sidebar on the left, touch product grid in the middle, and persistent live order tray on the right.
> • **No manual toggle button:** It responds dynamically to screen resizing or device rotation without user intervention.

6. Anticipated Panel Questions & Ready Answers (Cheat Sheet)

Q1: "Why did you use Capacitor instead of building separate native apps from scratch?"

Answer: "Capacitor allows us to maintain a single unified codebase with 100% synchronized business logic. This drastically lowers development and maintenance costs for small local coffee shops in Kabacan while still compiling into native Android APKs and iOS apps with full offline capabilities."

Q2: "What happens if the internet goes down in Kabacan?"

Answer: "The native Android APK runs 100% locally on the device hardware using local database persistence. It does not depend on cloud internet or a host PC to browse the menu, customize drinks, or issue cash order queue tickets."

Q3: "How does the inventory system prevent running out of stock?"

Answer: "Every menu item is mapped to recipe ingredient amounts (e.g. 0.02kg coffee beans, 0.2L milk). When an order is placed, stock is automatically subtracted. When any ingredient drops to or below its minimum threshold, the Admin Dashboard immediately raises a red Low-Stock Alert banner."

Q4: "How does the cashier verify cashless GCash or PayMaya payments?"

Answer: "The kiosk generates a dynamic QR code for GCash or PayMaya and asks the customer for the transaction reference number. The order enters the Cashier's POS screen with the reference number clearly highlighted for quick payment confirmation before moving to the barista."

Q5: "Why is there no button to switch to mobile view?"

Answer: "For a kiosk system, human customers should never have to manually adjust view settings. Our system utilizes a custom responsive hook that automatically detects viewport dimensions and device orientation, switching layouts seamlessly behind the scenes."